Solution: Daily Temperatures

Problem Statement

You are given a list of daily `temperatures`. Your task is to return an `answer` array such that `answer[i]` is the number of days you would have to wait until a warmer temperature for each of the days. If there is no future day for which this is possible, put 0 instead.

Examples

1. - **Input:** [45, 50, 40, 60, 55]
   - **Expected Output:** [1, 2, 1, 0, 0]
   - **Justification:** The next day after the first day is warmer (50 > 45). Two days after the second day, the temperature is warmer (60 > 50).. The next day after the third day is warmer (60 > 40). There are no warmer days after the fourth and fifth days.
2. - **Input:** [80, 75, 85, 90, 60]
   - **Expected Output:** [2, 1, 1, 0, 0]
   - **Justification:** Two days after the first day, the temperature is warmer (85 > 80). The next day after the second day is warmer (85 > 75). The next day after the third day is warmer (90 > 85). There are no warmer days after the fourth and fifth days.
3. - **Input:** [32, 32, 32, 32, 32]
   - **Expected Output:** [0, 0, 0, 0, 0]
   - **Justification:** All the temperatures are the same, so there are no warmer days ahead.

Solution

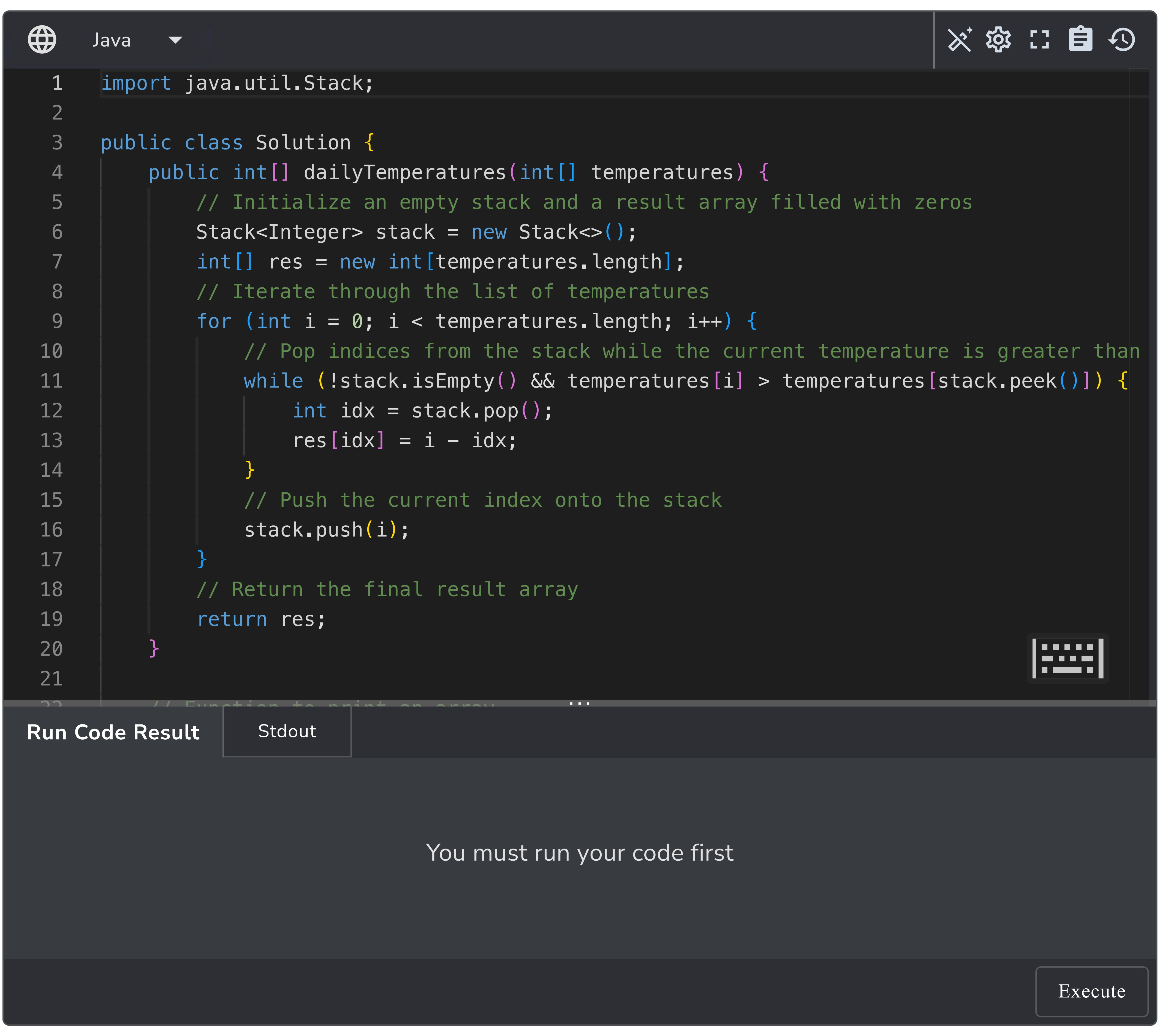
- **Understanding the Problem**:
  - The problem is to find the number of days until a warmer day for each day in the list.
- **Approach**:
  - Use a stack to keep track of the indices of the days.
  - Iterate through the list of temperatures.
    - While the stack is not empty and the current temperature is greater than the temperature at the index at the top of the stack:
      - Pop an index from the stack and calculate the difference between the current day and the popped index.
      - Store this difference at the popped index in the result array.
    - Push the current day's index onto the stack.
- **Why This Works**:
  - The stack keeps track of the days for which a warmer day has not yet been found.
  - When a warmer day is found, the days are popped from the stack and the difference in days is calculated.

Algorithm Walkthrough

Consider the input `[45, 50, 40, 60, 55]`:

- Initialize an empty stack and a result array filled with zeros: `stack = []`, `res = [0, 0, 0, 0, 0]`.
- Iterate through the list:
  - Day 1 (45): `stack = [0]`.
  - Day 2 (50): Pop `0` from `stack`, `res[0] = 1 - 0 = 1`, `stack = [1]`.
  - Day 3 (40): `stack = [1, 2]`.
  - Day 4 (60): Pop `2` from `stack`, `res[2] = 3 - 2 = 1`, Pop `1` from `stack`, `res[1] = 3 - 1 = 2`, `stack = [3]`.
  - Day 5 (55): `stack = [3, 4]`.
- End of iteration, `res = [1, 2, 1, 0, 0]`.

Code Development



```java
import java.util.Stack;

public class Solution {
    public int[] dailyTemperatures(int[] temperatures) {
        // Initialize an empty stack and a result array filled with zeros
        Stack<Integer> stack = new Stack<>();
        int[] res = new int[temperatures.length];
        // Iterate through the list of temperatures
        for (int i = 0; i < temperatures.length; i++) {
            // Pop indices from the stack while the current temperature is greater than
            while (!stack.isEmpty() && temperatures[i] > temperatures[stack.peek()]) {
                int idx = stack.pop();
                res[idx] = i - idx;
            }
            // Push the current index onto the stack
            stack.push(i);
        }
        // Return the final result array
        return res;
    }

```

You must run your code first

Complexity Analysis

- **Time Complexity:** O(n), where n is the number of temperatures. This is because the algorithm iterates through the list of temperatures once, performing constant-time operations for each temperature.
- **Space Complexity:** O(n), as it uses a stack to keep track of indices.

← Previous
Daily Temperatures (Medium)

Next →
Group Anagrams (Medium)

☐ Mark as Completed